

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Florea David, FAF-242

Report

Laboratory work nr. 2

Meet the OS You Will Build

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Setup

Track B (xv6). Ubuntu 26.04.1 LTS VM, cross-compiler `riscv64-linux-gnu-gcc`.

```
cd ~/xv6-riscv 2>/dev/null || \
  git clone https://github.com/mit-pdos/xv6-riscv ~/xv6-riscv
cd ~/xv6-riscv
git log -1 --format="%h %ad %s" --date=short
ls -l
riscv64-linux-gnu-gcc --version | head -1
```

```
Cloning into '/home/david/xv6-riscv'...
06aad25 2026-09-26 Add killzero test
-rw-rw-r-- 1 david david 1174 Oct  5 14:15 LICENSE
-rw-rw-r-- 1 david david 5744 Oct  5 14:15 Makefile
-rw-rw-r-- 1 david david 2441 Oct  5 14:15 README
drwxrwxr-x 2 david david 4096 Oct  5 14:15 kernel
drwxrwxr-x 2 david david 4096 Oct  5 14:15 mkfs
-rwxrwxr-x 1 david david 6147 Oct  5 14:15 test-xv6.py
drwxrwxr-x 2 david david 4096 Oct  5 14:15 user
riscv64-linux-gnu-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
```

First build (QEMU missing)

The compiler was already on the VM, but the emulator wasn't. The kernel and file system image still built, and only the final step failed:

```
make qemu
```

```
/bin/sh: 1: qemu-system-riscv64: not found
riscv64-linux-gnu-gcc -march=rv64gc -g -ffile-prefix-map=/home/david/xv6-riscv=. -c -o
     kernel/entry.o kernel/entry.S
...
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init ...
make: qemu-system-riscv64: No such file or directory
make: *** [Makefile:183: qemu] Error 127
```

After installing QEMU:

```
qemu-system-riscv64 --version | head -1
```

```
QEMU emulator version 10.2.1 (Debian 1:10.2.1+ds-1ubuntu3.2)
```

Part 1 – Build and Boot

```
make qemu
```

```
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -
    nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format
    =raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
```

```
hart 2 starting
init: starting sh
$
```

QEMU emulates a RISC-V `virt` machine with 3 cores (`-smp 3`, hence harts 1 and 2 starting next to hart 0) and 128 MB of RAM, and uses `fs.img` as its disk. Ctrl-A, X exits the emulator.

Part 2 – Using xv6

```
$ ls
```

```
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
sleep          2 5 35280
pingpong       2 6 36672
forktest       2 7 18136
grep           2 8 44184
init           2 9 35976
kill           2 10 35512
ln             2 11 35304
ls             2 12 42880
mkdir          2 13 35568
rm             2 14 35552
sh             2 15 58312
stressfs       2 16 36416
usertests      2 17 206800
grind          2 18 51824
wc             2 19 37728
zombie         2 20 34848
logstress      2 21 37560
forphan        2 22 36352
dorphan        2 23 35800
sync           2 24 34944
console        3 25 0
```

The columns are name, type (1 = directory, 2 = file, 3 = device), inode number and size in bytes.

```
$ cat README
$ echo hello xv6
$ ls | grep c
$ wc README
```

```
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.
...
hello xv6
cat            2 3 36728
echo           2 4 35592
wc             2 19 37728
```

```
sync           2 24 34944
console        3 25 0
48 336 2441 README
```

```
$ usertests -q
```

```
usertests starting
test copyin: OK
test copyout: OK
...
test kernmem: usertrap(): unexpected scause 0xd pid=5593
...
test lazy_sbrk: OK
test partial_write: OK
test unlinkcwd: OK
ALL TESTS PASSED
```

Observations

(1) Three programs xv6 ships with. `ls`, `cat` and `echo` (also `grep`, `wc`, `sh`, `kill`, `mkdir`, `rm`, `usertests`, ...). `ls` listed 22 programs in the root directory, plus `README` and the `console` device.

(2) What a pipe needs. For `ls | grep c` to work the OS must provide:

- **process creation** (`fork` + `exec`): the shell starts `ls` and `grep` as two separate processes running at the same time;
- **inter-process communication** (the `pipe` system call plus file descriptors and `dup`): a kernel buffer connects the stdout of `ls` to the stdin of `grep`.

(3) xv6 shell vs. Linux shell. The xv6 `sh` handles the same basics as bash (running programs, `|`, `</>`, `;`, `&`), but it is a 499-line program with no variables, job control, history, tab completion or scripting.

Interesting output: `usertests -q` passed all tests, but along the way it printed lines like
`usertrap(): unexpected scause 0xd`. Those come from tests such as `kernmem` and `stacktest` that deliberately make a bad memory access. They check that the kernel kills only the faulting process and keeps running.

Part 3 – Reading the Source

```
sed -n "1,30p" user/cat.c
```

```
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
```

```
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}
```

```
cat user/user.h          # system-call part
```

```
// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
...
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);
```

```
grep -n "sys_read\|sys_write" kernel/sysfile.c | head
grep -n -B1 -A12 "^sys_read" kernel/sysfile.c
```

```
69:sys_read(void)
83:sys_write(void)
68-uint64
69:sys_read(void)
70-{
71-  struct file *f;
72-  int n;
73-  uint64 p;
74-
75-  argaddr(1, &p);
76-  argint(2, &n);
77-  if (argfd(0, 0, &f) < 0)
78-    return -1;
79-  return fileread(f, p, n);
80-}
```

```
grep -n "sys_read\|sys_write\|sys_pause\|SYS_read\|SYS_write\|SYS_pause" kernel/
    syscall.c kernel/syscall.h
```

```
kernel/syscall.c:88:extern uint64 sys_read(void);
kernel/syscall.c:96:extern uint64 sys_pause(void);
kernel/syscall.c:99:extern uint64 sys_write(void);
kernel/syscall.c:115:  [SYS_read]    = sys_read,
kernel/syscall.c:123:  [SYS_pause]   = sys_pause,
kernel/syscall.c:126:  [SYS_write]   = sys_write,
kernel/syscall.h:6:#define SYS_read   5
kernel/syscall.h:14:#define SYS_pause  13
kernel/syscall.h:17:#define SYS_write  16
```

Observations

(1) System calls in `user/cat.c`.

call	asks the kernel to...
`open(path, O_RDONLY)`	open the named file and return a file descriptor
`read(fd, buf, n)`	copy up to `n` bytes from the file into the buffer
`write(1, buf, n)`	copy `n` bytes from the buffer to fd 1 (stdout)
`close(fd)`	release the file descriptor
`exit(status)`	end the process and report a status to its parent

`fprintf` is not a system call. It's a library function from `user/printf.c` that ends up calling `write`.

(2) Where `sys_read` is implemented. In `kernel/sysfile.c`, **line 69** (the function body is lines 69–80; `sys_write` is at line 83). The user-side stub generated from `user/usys.pl` puts the number `SYS_read` = 5 in a register and executes `ecall`. The kernel's table in `kernel/syscall.c` (line 115) then maps number 5 to `sys_read`, which fetches the arguments and calls `fileread`.

(3) `kernel/` vs. `user/`. `kernel/` is the OS itself and runs in privileged (supervisor) mode with direct access to hardware and all memory. `user/` contains ordinary programs that run unprivileged and can only get things done by asking the kernel through system calls.

Interesting output: the whole OS is small. `kernel/` has 44 source files with 6 605 lines, and `user/` has 26 files with 5 635 lines (almost two thirds of those, 3 538, are the test suite `usertests.c`).

Part 4 – sleep

Current xv6 renamed the `sleep` system call to `pause`, because the kernel already has its own internal `sleep()` for waiting on events. `user/user.h` declares `int pause(int);`, and the kernel side is `sys_pause` in `kernel/sysproc.c`. So my program calls `pause()` instead of the handout's `sleep()`.

Creating `user/sleep.c`:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
```

```
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // system call into the kernel (sleep() was renamed pause()
    in current xv6)
  exit(0);
}
```

Adding `sleep` (and `pingpong`) to `UPROGS` in the `Makefile`:

```
sed -i 's|^\t\$U/_echo\\$|&\n\t$U/_sleep\\\n\t$U/_pingpong\\|' Makefile
grep -n "_echo\|_sleep\|_pingpong" Makefile | cat -A
```

```
134:^I$U/_echo\$
135:^I$U/_sleep\$
136:^I$U/_pingpong\$
```

(`cat -A` shows the tab as `^I` and the line end as `$`, confirming the tab indentation and the trailing backslash.)

Rebuilding:

```
make kernel/kernel fs.img
```

```
riscv64-linux-gnu-gcc ... -c -o user/sleep.o user/sleep.c
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_sleep user/sleep.o
     user/ulib.o user/usys.o user/printf.o user/umalloc.o
...
mkfs/mkfs fs.img README user/_cat user/_echo user/_sleep user/_pingpong user/_forktest
     ...
```

Running `sleep` in QEMU (each output line is timestamped on the host):

```
$ sleep
$ sleep abc
$ echo before
$ sleep 10
$ echo after
$ echo start; sleep 50; echo done
```

```
[14:38:22.267] usage: sleep <ticks>
[14:38:26.186] before
[14:38:27.102] after
[14:39:07.160] start
[14:39:12.251] done
```

Without an argument the program prints its usage message and exits. `sleep 10` paused about 0.92 s and `sleep 50` about 5.09 s, so one xv6 timer tick is about 100 ms. `sleep abc` returns immediately, because `atoi("abc")` is 0.

Optional – pingpong

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
```

```
{
  int p2c[2], c2p[2];   // parent->child and child->parent pipes
  char b = 'x';

  if(pipe(p2c) < 0 || pipe(c2p) < 0){
    fprintf(2, "pingpong: pipe failed\n");
    exit(1);
  }

  int pid = fork();
  if(pid < 0){
    fprintf(2, "pingpong: fork failed\n");
    exit(1);
  }

  if(pid == 0){
    close(p2c[1]);
    close(c2p[0]);
    if(read(p2c[0], &b, 1) != 1){
      fprintf(2, "pingpong: child read failed\n");
      exit(1);
    }
    printf("%d: received ping\n", getpid());
    if(write(c2p[1], &b, 1) != 1){
      fprintf(2, "pingpong: child write failed\n");
      exit(1);
    }
    close(p2c[0]);
    close(c2p[1]);
    exit(0);
  }

  close(p2c[0]);
  close(c2p[1]);
  if(write(p2c[1], &b, 1) != 1){
    fprintf(2, "pingpong: parent write failed\n");
    exit(1);
  }
  if(read(c2p[0], &b, 1) != 1){
    fprintf(2, "pingpong: parent read failed\n");
    exit(1);
  }
  printf("%d: received pong\n", getpid());
  close(p2c[1]);
  close(c2p[0]);
  wait(0);
  exit(0);
}
```

```
$ pingpong
10: received ping
9: received pong
```

The child (PID 10) blocks in `read` until the parent (PID 9) writes the byte, so "ping" is always printed before "pong".

Conclusion

In this laboratory work I built xv6 from source with the RISC-V cross-compiler and booted it in QEMU on an emulated 3-core RISC-V machine. Inside it I used the familiar Unix tools (`ls`, `cat`, `echo`, `wc`) and a pipe, and the kernel test suite passed. Reading the source showed the split at the heart of an OS. `user/cat.c` only calls `read` and `write`; a stub turns each call into a numbered `ecall`, and `kernel/syscall.c` dispatches it to `sys_read` in `kernel/sysfile.c`. Finally I wrote my own user program, `sleep`, which uses the system call now named `pause`, and the optional `pingpong`, which uses `fork` and two pipes. Both run correctly in the kernel I compiled myself.